ADITYA

ENLIGHTENS THE NESCIENCE

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

ADB ROAD, SURAMPALEM, E.G. DIST.

DEPARTMENT OF

Name: PIN No. | | | | | | | | | | |

Certified that this is the bonafide record of practical work done by

Mr. / Ms. .....

a students of ..... with PIN No.....

in the ..... Laboratory during the year .....

No of Practicals Conducted [ ] No. of Practicals Attended [ ]

Signature - Faculty Incharge Signature - Head of the Department

Submitted for the Practical examination held on.....

EXAMINER - 1 EXAMINER - 2

Week 1

Execute all DDL, DML and DCL commands on sample tables.

DDL (Data Definition Language):

1. CREATE TABLE: This command is used to create a new table in the database. It specifies the table name and defines its structure, including column names, data types, and constraints. In the example, we created two tables: **employees** and **departments**, each with their respective columns.

DDL (Data Definition Language):

```
-- Creating a sample table
CREATE TABLE employees (
employee_id INT PRIMARY KEY,
first_name VARCHAR(50),
last_name VARCHAR(50),
department_id INT
);

-- Creating another sample table
CREATE TABLE departments (
department_id INT PRIMARY KEY,
department_name VARCHAR(50)
);
```

DML (Data Manipulation Language):

1. **INSERT INTO:** This command is used to insert new rows of data into a table. It specifies the table name and the values to be inserted into each column for each row. In the example, we inserted data into the **employees** and **departments** tables, populating them with sample information.

```
-- Inserting data into the employees table
INSERT INTO employees (employee_id, first_name, last_name, department_id)
VALUES (1, 'John', 'Doe', 1001),
(2, 'Jane', 'Smith', 1002),
(3, 'Alice', 'Johnson', 1001);

-- Inserting data into the departments table
INSERT INTO departments (department_id, department_name)
VALUES (1001, 'Engineering'),
(1002, 'Marketing');
```

DCL (Data Control Language):

1. **GRANT:** This command is used to grant specific privileges or permissions to a user or role. In this example, we granted the **SELECT** privilege on the **employees** table to a user named **user_name**. This means that the specified user now has permission to retrieve data from the **employees** table.

2. **REVOKE:** This command is used to revoke previously granted privileges or permissions from a user or role. In this example, we revoked the **INSERT** privilege on the **departments** table from a user named **user_name**. This means that the specified user no longer has permission to insert data into the **departments** table.

```
-- Granting SELECT permission on the employees table to a user
GRANT SELECT ON employees TO user_name;

-- Revoking INSERT permission on the departments table from a user
REVOKE INSERT ON departments FROM user_name;
```

These commands are fundamental for managing the structure, data, and access control of a database. They allow you to define tables, manipulate data within those tables, and control who has permission to perform certain actions on the database objects.

Week 2

Implementation of different types of operators and built-in functions with suitable examples.

Here are examples of different types of operators and built-in functions commonly used in SQL:

1. Arithmetic Operators:

```
-- Addition
SELECT 5 + 3;

-- Subtraction
SELECT 10 - 7;

-- Multiplication
SELECT 4 * 6;

-- Division
SELECT 20 / 4;

-- Modulus (Remainder)
SELECT 15 % 4;
```

---

2. Comparison Operators:

```
-- Equal to
SELECT * FROM employees WHERE department_id = 1001;

-- Not equal to
SELECT * FROM employees WHERE department_id <> 1001;

-- Greater than
SELECT * FROM employees WHERE employee_id > 2;

-- Less than
SELECT * FROM employees WHERE employee_id < 5;

-- Greater than or equal to
SELECT * FROM employees WHERE department_id >= 1001;
```

```
-- Less than or equal to
SELECT * FROM employees WHERE department_id <= 1002;
```

---

3. Logical Operators:

```
-- AND
SELECT * FROM employees WHERE department_id = 1001 AND first_name = 'John';


-- OR
SELECT * FROM employees WHERE department_id = 1001 OR department_id = 1002;

-- NOT
SELECT * FROM employees WHERE NOT department_id = 1001;
```

---

4. String Functions:

```
-- CONCAT
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM employees;

-- SUBSTRING
SELECT SUBSTRING(first_name, 1, 3) AS initials FROM employees;

-- UPPER
SELECT UPPER(last_name) AS last_name_upper FROM employees;

-- LOWER
SELECT LOWER(first_name) AS first_name_lower FROM employees;
```

---

5. Mathematical Functions:

```
-- ABS
SELECT ABS(-10) AS absolute_value;

-- SQRT
SELECT SQRT(25) AS square_root;
```

```
-- POWER
SELECT POWER(2, 3) AS power;

-- ROUND
SELECT ROUND(3.14159, 2) AS rounded_pi;
```

---

6. Aggregate Functions:

```
-- COUNT

SELECT COUNT(*) AS total_employees FROM employees;


-- SUM

SELECT SUM(salary) AS total_salary FROM employees;


-- AVG

SELECT AVG(salary) AS average_salary FROM employees;


-- MAX

SELECT MAX(salary) AS max_salary FROM employees;


-- MIN

SELECT MIN(salary) AS min_salary FROM employees;
```

Week 3

Implementation of different types of joins with suitable examples.

Here are examples of different types of joins commonly used in SQL:

1. INNER JOIN:

SELECT employees.employee_id, employees.first_name, employees.last_name, departments.department_name

FROM employees

INNER JOIN departments ON employees.department_id = departments.department_id;

2. LEFT JOIN (or LEFT OUTER JOIN):

SELECT employees.employee_id, employees.first_name, employees.last_name, departments.department_name

FROM employees

LEFT JOIN departments ON employees.department_id = departments.department_id;

3. RIGHT JOIN (or RIGHT OUTER JOIN):

SELECT employees.employee_id, employees.first_name, employees.last_name, departments.department_name

FROM employees

RIGHT JOIN departments ON employees.department_id = departments.department_id;

4. FULL JOIN (or FULL OUTER JOIN):

```
SELECT employees.employee_id, employees.first_name, employees.last_name,
departments.department_name

FROM employees

FULL JOIN departments ON employees.department_id = departments.department_id;
```

5. CROSS JOIN:

```
SELECT employees.employee_id, employees.first_name, employees.last_name,
departments.department_name

FROM employees

CROSS JOIN departments;
```

These examples demonstrate different types of joins in SQL, including INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, and CROSS JOIN. Depending on your specific requirements and the structure of your tables, you can choose the appropriate join type to retrieve the desired data.

Week 4

Create views, partitions, Sequence, Indexes and locks for a particular DB.

Let's go through creating views, partitions, sequences, indexes, and locks for a particular database. I'll provide examples for each:

1. Views:

```
CREATE VIEW view_name AS

SELECT column1, column2

FROM table_name

WHERE condition;
```

Example:

```
CREATE VIEW employee_details AS

SELECT employee_id, first_name, last_name, salary

FROM employees

WHERE department_id = 1001;
```

2. Partitions:

```
CREATE TABLE sales (

sales_id INT,

sales_date DATE,

amount DECIMAL

)

PARTITION BY RANGE (YEAR(sales_date)) (

PARTITION p2019 VALUES LESS THAN (2020),

PARTITION p2020 VALUES LESS THAN (2021),

PARTITION p2021 VALUES LESS THAN (2022),
```

```
PARTITION p2022 VALUES LESS THAN MAXVALUE
);
```

3. Sequence:

```
CREATE SEQUENCE order_sequence
START WITH 1000
INCREMENT BY 1
MINVALUE 1000
MAXVALUE 9999
CYCLE;
```

4. Indexes:

```
CREATE INDEX idx_employee_department ON employees (department_id);
```

5. Locks:

```
LOCK TABLE employees IN EXCLUSIVE MODE;
```

These are just examples, and the syntax may vary depending on the database management system you are using (e.g., MySQL, PostgreSQL, Oracle). Make sure to adjust the examples according to your specific requirements and database system. Additionally, be cautious when applying locks, as they can impact the concurrency and performance of your database operations.

Week 5

Implement different types of constraints on relations.

Constraints are rules enforced on data within a table to maintain data integrity. Here are examples of different types of constraints that can be applied to relations (tables) in a database:

1. NOT NULL Constraint:

```
CREATE TABLE employees (
employee_id INT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
department_id INT
);
```

In this example, **first_name** and **last_name** columns must have a value, and NULL values are not allowed.

---

2. UNIQUE Constraint:

```
CREATE TABLE departments (
department_id INT PRIMARY KEY,
department_name VARCHAR(50) UNIQUE
);
```

In this example, the **department_name** column must contain unique values.

---

3. PRIMARY KEY Constraint:

```
CREATE TABLE products (
product_id INT PRIMARY KEY,
product_name VARCHAR(50),
price DECIMAL(10, 2)
```

```
);
```

In this example, **product_id** uniquely identifies each row in the table, making it the primary key.

---

4. FOREIGN KEY Constraint:

```
CREATE TABLE orders (
order_id INT PRIMARY KEY,
product_id INT,
quantity INT,
FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

In this example, **product_id** in the **orders** table is a foreign key that references the **product_id** column in the **products** table, establishing a relationship between the two tables.

---

5. CHECK Constraint:

```
CREATE TABLE students (
student_id INT PRIMARY KEY,
age INT CHECK (age >= 18)
);
```

In this example, the **age** column must have a value greater than or equal to 18 due to the check constraint.

---

6. DEFAULT Constraint:

```
CREATE TABLE messages (
message_id INT PRIMARY KEY,
message_text VARCHAR(255),
sent_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

In this example, if no value is provided for **sent_date** when inserting a row, the current timestamp will be used as the default value.

These examples demonstrate how various constraints can be applied to relations in a database to enforce data integrity and structure.

Week 6

Implementation of sub-queries and nested queries.

Subqueries (also known as nested queries or inner queries) are queries that are nested inside another query. They are commonly used to retrieve data based on the results of another query. Here are examples of subqueries and nested queries:

1. Subquery in WHERE clause:

```
SELECT first_name, last_name

FROM employees

WHERE department_id = (

SELECT department_id

FROM departments

WHERE department_name = 'Engineering'

);
```

In this example, the inner query retrieves the **department_id** for the department with the name 'Engineering', and then the outer query selects **first_name** and **last_name** of employees who belong to that department.

---

2. Subquery in SELECT clause:

```
SELECT first_name, last_name,

(SELECT MAX(salary) FROM salaries WHERE salaries.employee_id = employees.employee_id) AS max_salary

FROM employees;
```

In this example, the subquery calculates the maximum salary for each employee (**MAX(salary)**), and the outer query selects **first_name**, **last_name**, and the calculated **max_salary** for each employee.

---

3. Subquery in FROM clause (Inline View):

```
SELECT department_name, total_employees
FROM (
SELECT department_name, COUNT(*) AS total_employees
FROM employees e
JOIN departments d ON e.department_id = d.department_id
GROUP BY department_name
) AS department_summary;
```

In this example, the subquery calculates the total number of employees (COUNT(*)) in each department, and the outer query selects department_name and total_employees from the result of the subquery.

---

4. Subquery with EXISTS:

```
SELECT first_name, last_name
FROM employees
WHERE EXISTS (
SELECT 1
FROM orders
WHERE orders.employee_id = employees.employee_id
);
```

In this example, the subquery checks if there are any orders associated with each employee (**EXISTS**), and the outer query selects **first_name** and **last_name** of employees who have at least one order.

These are just a few examples of how subqueries and nested queries can be used in SQL to perform complex data retrieval and analysis tasks. They are powerful tools for querying data based on conditions derived from other queries.

Week 7

Implement Queries on Group By & Having Clauses, ALIAS, Sequence By, Order By.

Here are examples of SQL queries that involve the GROUP BY, HAVING, ALIAS, ORDER BY, and SEQUENCE BY clauses:

1. GROUP BY and HAVING clauses:

```
SELECT department_id, COUNT(*) AS total_employees
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

This query calculates the total number of employees in each department (**COUNT(*)**) and groups them by **department_id**. Then, it uses the **HAVING** clause to filter out departments with more than 5 employees.

---

2. ALIAS usage:

```
SELECT first_name AS "First Name", last_name AS "Last Name"
FROM employees;
```

In this query, **first_name** and **last_name** columns are aliased as "First Name" and "Last Name" respectively, providing more descriptive column names in the result set.

---

3. ORDER BY clause:

```
SELECT employee_id, first_name, last_name, salary
FROM employees
ORDER BY salary DESC;
```

This query retrieves **employee_id**, **first_name**, **last_name**, and **salary** columns from the **employees** table, ordered by **salary** in descending order (**DESC**).

---

4. SEQUENCE BY clause:

SELECT product_id, product_name, price

FROM products

SEQUENCE BY price DESC;

This query selects **product_id**, **product_name**, and **price** columns from the **products** table, ordering the result set by **price** in descending order using the **SEQUENCE BY** clause.

These are just examples of how to use these clauses in SQL queries. Each clause serves a specific purpose, allowing you to manipulate and analyze data effectively.

Week 8

Control Structure.

a) Write a PL/SQL block for Addition of Two Numbers.

Here's a PL/SQL block for adding two numbers:

```
SET SERVEROUTPUT ON;


DECLARE
num1 NUMBER := 10;
num2 NUMBER := 20;
sum NUMBER;
BEGIN
sum := num1 + num2;
DBMS_OUTPUT.PUT_LINE('The sum of ' || num1 || ' and ' || num2 || ' is: ' || sum);
END;
/
```

This PL/SQL block declares two variables **num1** and **num2** initialized with the values 10 and 20 respectively. It then calculates the sum of **num1** and **num2** and stores it in the **sum** variable. Finally, it uses the **DBMS_OUTPUT.PUT_LINE** procedure to display the result.

Sample Output:
The sum of 10 and 20 is: 30

This PL/SQL block adds two numbers and prints the result using **DBMS_OUTPUT.PUT_LINE.**

B) b) **Write a PL/SQL block for IF, IF and else condition.**
Here's a PL/SQL block demonstrating IF and IF-ELSE conditions:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
num1 NUMBER := 10;
num2 NUMBER := 20;
BEGIN
-- IF condition
IF num1 > num2 THEN
DBMS_OUTPUT.PUT_LINE(num1 || ' is greater than ' || num2);
END IF;

-- IF-ELSE condition
IF num1 < num2 THEN
DBMS_OUTPUT.PUT_LINE(num1 || ' is less than ' || num2);
ELSE
DBMS_OUTPUT.PUT_LINE(num1 || ' is not less than ' || num2);
END IF;
END;
/
```

In this PL/SQL block:

- The variables **num1** and **num2** are declared and initialized with values 10 and 20 respectively.
- The first IF condition checks if **num1** is greater than **num2**. If true, it prints a message indicating that.
- The second IF-ELSE condition checks if **num1** is less than **num2**. If true, it prints a message indicating that. If false, it prints a message indicating that **num1** is not less than **num2**.

Sample Output:

10 is less than 20

10 is not less than 20

This PL/SQL block demonstrates the usage of IF and IF-ELSE conditions to control the flow of execution based on specified conditions.

C) **Write a PL/SQL block for the implementation of loops.**

Here's a PL/SQL block demonstrating the implementation of loops:

SET SERVEROUTPUT ON;

```
DECLARE
i NUMBER := 1;
BEGIN
-- WHILE loop
WHILE i <= 5 LOOP
DBMS_OUTPUT.PUT_LINE('Value of i: ' || i);
i := i + 1;
END LOOP;

-- FOR loop
FOR j IN 1..5 LOOP
DBMS_OUTPUT.PUT_LINE('Value of j: ' || j);
END LOOP;
END;
/
```

In this PL/SQL block:

- The variable **i** is initialized with the value 1.
- The WHILE loop iterates as long as the value of **i** is less than or equal to 5. Inside the loop, the value of **i** is printed, and then incremented by 1 in each iteration.
- The FOR loop iterates from 1 to 5. Inside the loop, the value of the loop index **j** is printed.

Sample Output:

Value of i: 1

Value of i: 2

Value of i: 3

Value of i: 4

Value of i: 5

Value of j: 1

Value of j: 2

Value of j: 3

Value of j: 4

Value of j: 5

This PL/SQL block demonstrates the usage of WHILE and FOR loops to iterate over a sequence of statements a specified number of times.

D) Write a PL/SQL block for greatest of three numbers using IF and ELSEIF.

Here's a PL/SQL block for finding the greatest of three numbers using IF and ELSEIF statements:

```
SET SERVEROUTPUT ON;
DECLARE
num1 NUMBER := 10;
num2 NUMBER := 20;
num3 NUMBER := 15;
greatest NUMBER;
BEGIN
IF num1 >= num2 AND num1 >= num3 THEN
greatest := num1;
ELSIF num2 >= num1 AND num2 >= num3 THEN
greatest := num2;
ELSE
greatest := num3;
END IF;

DBMS_OUTPUT.PUT_LINE('The greatest number among ' || num1 || ', ' || num2 || ', and ' || num3 || ' is: '
|| greatest);
END;
/
```

In this PL/SQL block:

- The variables num1, num2, and num3 are initialized with values 10, 20, and 15 respectively.
- The IF statement checks three conditions: if num1 is greater than or equal to num2 and num3, if num2 is greater than or equal to num1 and num3, and if neither of the previous conditions are true (i.e., num3 is the greatest).
- Depending on the condition met, the greatest number is assigned to the variable greatest.
- Finally, the result is printed using DBMS_OUTPUT.PUT_LINE.

Sample Output:

The greatest number among 10, 20, and 15 is: 20

This PL/SQL block finds the greatest of three numbers using IF and ELSEIF statements and prints the result.

Week 9

Exception Handling- Implement the following with respect to exception handling. Raising Exceptions, User Defined Exceptions, Pre-Defined Exceptions.

Let's implement each aspect of exception handling with sample outputs:

1. Raising Exceptions:

```
DECLARE
v_num1 NUMBER := 10;
v_num2 NUMBER := 0;
BEGIN
IF v_num2 = 0 THEN
RAISE ZERO_DIVIDE;
ELSE
DBMS_OUTPUT.PUT_LINE('Result: ' || v_num1 / v_num2);
END IF;
EXCEPTION
WHEN ZERO_DIVIDE THEN
DBMS_OUTPUT.PUT_LINE('Error: Division by zero');
END;
/
```

Sample Output (Division by zero):

Error: Division by zero

---

2. User-Defined Exceptions:

```
DECLARE
v_age NUMBER := 15;
v_min_age CONSTANT NUMBER := 18;
BEGIN
IF v_age < v_min_age THEN
RAISE_APPLICATION_ERROR(-20001, 'Age must be at least 18');
ELSE
DBMS_OUTPUT.PUT_LINE('You are eligible');
END IF;
EXCEPTION
```

```
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Sample Output (Age validation):

Error: Age must be at least 18

---

3. Pre-Defined Exceptions:

```
DECLARE
v_salary NUMBER := -1000;
BEGIN
IF v_salary < 0 THEN
RAISE VALUE_ERROR;
ELSE
DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
END IF;
EXCEPTION
WHEN VALUE_ERROR THEN
DBMS_OUTPUT.PUT_LINE('Error: Invalid salary value');
END;
/
```

Sample Output (Invalid salary value):

Error: Invalid salary value

These examples demonstrate the implementation of different aspects of exception handling in PL/SQL, including raising predefined and user-defined exceptions, and handling exceptions with appropriate error messages.

Week- 10

Write PL/SQL block for an application using exception handling

Procedures.

a) Write a PL/SQL Procedure using Positional Parameters.

Answer)

Explanation:

Let's create a PL/SQL procedure that uses positional parameters and includes exception handling. Positional parameters allow us to pass values to a procedure without explicitly naming them.

In this corrected example:

- We define a procedure called process_student that takes three positional parameters: p_student_id, p_student_name, and p_marks.
- It checks if the provided student ID already exists in the students table.
- If the student ID exists, it raises a custom exception with an error message.
- Otherwise, it inserts the student record into the table and commits the transaction.

Remember to replace the table name (students) and column names with your actual database schema. Additionally, handle exceptions specific to your application requirements.

```
-- PL/SQL Procedure with Positional Parameters and Exception Handling
CREATE OR REPLACE PROCEDURE process_student(
p_student_id NUMBER,
p_student_name VARCHAR2,
p_marks NUMBER
) AS
v_student_count NUMBER;
BEGIN
-- Check if the student ID already exists
SELECT COUNT(*) INTO v_student_count
FROM students
WHERE student_id = p_student_id;

IF v_student_count > 0 THEN
-- Student ID already exists, raise an exception
```

```
RAISE_APPLICATION_ERROR(-20001, 'Student ID ' || p_student_id || ' already exists.');
ELSE
-- Insert the new student record
INSERT INTO students(student_id, student_name, marks)
VALUES (p_student_id, p_student_name, p_marks);
COMMIT;
DBMS_OUTPUT.PUT_LINE('Student record inserted successfully.');
END IF;
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

B)Write a PL/SQL Procedure using notational parameters.

Let's create a PL/SQL procedure that uses **named parameters** (also known as notational parameters). Named parameters allow us to pass values to a procedure by explicitly specifying the parameter names, rather than relying on their positions. This approach enhances code readability and makes it easier to understand the purpose of each parameter.

Here's an example of a PL/SQL procedure using named parameters:

In this example:

- We define a procedure called process_student that takes three named parameters: p_student_id, p_student_name, and p_marks.
- The parameter names are explicitly specified when calling the procedure.
- It checks if the provided student ID already exists in the students table.
- If the student ID exists, it raises a custom exception with an error message.
- Otherwise, it inserts the student record into the table and commits the transaction.

Remember to replace the table name (students) and column names with your actual database schema. Additionally, handle exceptions specific to your application requirements

```
-- PL/SQL Procedure with Named Parameters

CREATE OR REPLACE PROCEDURE process_student(
```

```
p_student_id NUMBER,

p_student_name VARCHAR2,

p_marks NUMBER

) AS

v_student_count NUMBER;

BEGIN

-- Check if the student ID already exists

SELECT COUNT(*) INTO v_student_count

FROM students

WHERE student_id = p_student_id;



IF v_student_count > 0 THEN

-- Student ID already exists, raise an exception

RAISE_APPLICATION_ERROR(-20001, 'Student ID ' || p_student_id || ' already exists.');

ELSE

-- Insert the new student record

INSERT INTO students(student_id, student_name, marks)

VALUES (p_student_id, p_student_name, p_marks);

COMMIT;

DBMS_OUTPUT.PUT_LINE('Student record inserted successfully.');

END IF;

EXCEPTION
```

```
WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;

/
```

C.Write a PL/SQL Procedure for GCD Numbers.

Let's create a PL/SQL procedure to find the Greatest Common Divisor (GCD) of two numbers using the Euclidean algorithm. The GCD represents the largest positive integer that divides both numbers without leaving a remainder.

Here's an example of a PL/SQL procedure that calculates the GCD of two given numbers:

Explanation:

In this example:

- We define a procedure called calculate_gcd that takes two input parameters: p_num1 and p_num2.
- If p_num1 is smaller than p_num2, we swap their values to ensure that p_num1 is the larger number.
- We use the Euclidean algorithm to calculate the GCD iteratively.
- The result is displayed using DBMS_OUTPUT.PUT_LINE.

You can call this procedure with your desired input values. Remember to replace the parameter names and adapt it to your specific use case.

```
-- PL/SQL Procedure to Calculate GCD
CREATE OR REPLACE PROCEDURE calculate_gcd(
p_num1 IN NUMBER,
p_num2 IN NUMBER
) AS
v_gcd NUMBER;
v_temp NUMBER;
BEGIN
-- Ensure that p_num1 is the bigger number
```

```
IF p_num1 < p_num2 THEN
v_temp := p_num1;
p_num1 := p_num2;
p_num2 := v_temp;
END IF;

-- Calculate GCD using the Euclidean algorithm
WHILE MOD(p_num1, p_num2) != 0 LOOP
v_temp := MOD(p_num1, p_num2);
p_num1 := p_num2;
p_num2 := v_temp;
END LOOP;

-- Display the result
DBMS_OUTPUT.PUT_LINE('GCD of ' || p_num1 || ' and ' || p_num2 || ' is ' || p_num1);
END;
/
```

D) Write a PL/SQL Procedures for cursor implementation (explicit and implicit cursors).

Let's dive into PL/SQL cursors and explore both **implicit** and **explicit** cursors.

1. **Implicit Cursors**:
   - An implicit cursor is automatically created by Oracle when executing certain SQL statements (such as SELECT INTO, INSERT, UPDATE, and DELETE).
   - Oracle internally manages the execution cycle of implicit cursors and provides information and statuses such as SQL%ROWCOUNT, SQL%ISOPEN, SQL%FOUND, and SQL%NOTFOUND.
   - However, implicit cursors are not elegant when the query returns zero or multiple rows, leading to exceptions like NO_DATA_FOUND or TOO_MANY_ROWS.
2. **Explicit Cursors**:
   - An explicit cursor is explicitly declared in the declaration section of a PL/SQL block or a package specification.
   - As a programmer, you have control over its execution cycle, including opening, fetching, and closing.
   - Here's how you work with explicit cursors:

Explanation:

1. In this example:

- We declare an explicit cursor named c_students that retrieves student data from the students table.
- We define a record variable (v_student_rec) to hold the fetched data.
- The OPEN statement opens the cursor, and the FETCH statement retrieves rows one by one.
- The loop processes each row until no more rows are found (%NOTFOUND).
- Finally, we close the cursor using the CLOSE statement.

Remember to adapt the cursor name (c_students) and query to your specific use case.

```
-- Example of an explicit cursor

DECLARE

CURSOR c_students IS

SELECT student_id, student_name, marks

FROM students;

-- Define the cursor query


v_student_rec c_students%ROWTYPE;

-- Declare a record variable to hold fetched data


BEGIN

OPEN c_students;

-- Open the cursor

LOOP

FETCH c_students INTO v_student_rec;

-- Fetch a row into the record variable
```

```
EXIT WHEN c_students%NOTFOUND;

-- Exit the loop when no more rows

-- Process the data (e.g., display student details)

DBMS_OUTPUT.PUT_LINE('Student ID: ' || v_student_rec.student_id ||

', Name: ' || v_student_rec.student_name ||

', Marks: ' || v_student_rec.marks);

END LOOP;

CLOSE c_students;

-- Close the cursor

END;
```

Week- 11

a) Write a PL/SQL block to implement factorial using functions.

Let's create a PL/SQL function to calculate the factorial of a given number. I'll provide you with an example implementation.

In this function:

- num is the input parameter representing the number for which we want to calculate the factorial.
- We initialize res_fact to 1.
- The loop iterates from 1 to num, multiplying res_fact by each value of i.
- Finally, we return the computed factorial value.

```
CREATE OR REPLACE FUNCTION factorial(num IN NUMBER) RETURN NUMBER IS

res_fact NUMBER := 1;

BEGIN

FOR i IN 1..num LOOP

res_fact := res_fact * i;

END LOOP;

RETURN res_fact;

END;
```

You can call this function using PL/SQL statements like this:

```
DECLARE

v_fact NUMBER;

BEGIN

v_fact := factorial(8);

DBMS_OUTPUT.PUT_LINE('Factorial of 8 = ' || v_fact);

END;
```

Sample Output:

The output will be the factorial of 8, which is 40320.

b) Write a PL/SQL function to search an address from the given database.

To create a PL/SQL function that searches for an address in a given database, we'll need to know more about the structure of your database and the specific table where the addresses are stored. However, I can provide you with a general example of how you might approach this task.

Let's assume you have a table called addresses with columns like street_address, city, state, and postal_code. We'll create a function that searches for an address based on a search term (e.g., street name, city, or postal code). Here's a simplified example:

Explanation:

In this function:

- p_search_term is the input parameter representing the search term.
- We use the LIKE operator to search for partial matches in the address fields.
- The result is returned as a cursor (SYS_REFCURSOR), which allows you to fetch the matching rows.

```
CREATE OR REPLACE FUNCTION search_address(p_search_term IN VARCHAR2) RETURN SYS_REFCURSOR IS

v_result SYS_REFCURSOR;

BEGIN

OPEN v_result FOR

SELECT street_address, city, state, postal_code

FROM addresses

WHERE street_address LIKE '%' || p_search_term || '%'

OR city LIKE '%' || p_search_term || '%'

OR postal_code LIKE '%' || p_search_term || '%';

RETURN v_result;

END;
```

/You can call this function from PL/SQL or SQL to retrieve the matching addresses. For example:

```
DECLARE
v_cursor SYS_REFCURSOR;
v_street VARCHAR2(100);
v_city VARCHAR2(50);
BEGIN
v_cursor := search_address('123 Main St');
LOOP
FETCH v_cursor INTO v_street, v_city;
EXIT WHEN v_cursor%NOTFOUND;
DBMS_OUTPUT.PUT_LINE('Street: ' || v_street || ', City: ' || v_city);
END LOOP;
CLOSE v_cursor;
END;
/
```

Week- 12

Write a DBMS program to prepare Pl/SQL reports for an application using functions.

To create PL/SQL reports for an application, you can leverage functions within your Oracle Reports. Let's break down the process:

1. **Create an External PL/SQL Library**:

- External PL/SQL libraries are modules that contain named PL/SQL functions and procedures. They can be stored in the database or in a file.
- These libraries allow you to reuse commonly-used PL/SQL constructs across reports, forms, and other Oracle products.
- You can create an external library using the Reports Builder. Launch Reports Builder and follow these steps:
  - Choose **File > New > Report**.
  - Select **Build a new report manually** and click **OK**.
  - Create the library by defining named PL/SQL functions and procedures.

2. **Create the Report Definition**:

- Manually create a query in the Data Model view.
- Define the layout of the report in the Paper Layout view.
- Add necessary report-level functions, formula columns, and other components.

3. **Example Scenario**:

- Let's assume you want to create a report that calculates total compensation for each employee.
- Here's a simplified example:

  - **Query**:
  - Retrieve employee data from the HR schema (e.g., employee ID, name, salary, bonus).
  - **Formula Column**:
  - Create a formula column that calculates the total compensation:

```
Sql>    total_compensation := salary + bonus;
```

Report-Level Function:

- Define a function that computes the total compensation for all employees

```
FUNCTION calculate_total_compensation RETURN NUMBER IS

total_comp NUMBER := 0;

BEGIN

SELECT SUM(salary + bonus) INTO total_comp FROM employees;

RETURN total_comp;

END;
```

Layout:

- Use the Report Block Wizard to design the layout.
- Add vertical space between records (e.g., every 10 records) for readability.

Format Trigger:

- Create a format trigger that uses the summary column (total compensation) and a user parameter (number of records per space).
- If you reach the specified number of records, insert a space.

1. **Run the Report**:

- Generate the report output, which will display employee data along with their total compensation.

Remember to adapt this example to your specific requirements, including the actual data model and layout.

Week- 13

Triggers:

a) Write a Trigger to pop-up the DML operations.

Let's dive into creating a DML (Data Manipulation Language) trigger. DML triggers in SQL Server allow you to respond to changes (inserts, updates, or deletes) made to a table. You can use them to perform additional actions based on these modifications. Here's how you can create DML triggers:

1. Syntax for Creating a DML Trigger:

- The basic syntax for creating a DML trigger is as follows:

CREATE [OR REPLACE] TRIGGER trigger_name

{BEFORE|AFTER} {INSERT|UPDATE|DELETE} ON table_name

Replace trigger_name with your desired trigger name, BEFORE or AFTER specifies when the trigger fires, and {INSERT|UPDATE|DELETE}.

Example: Change Tracking/Auditing Trigger (Update)

- Suppose we have a database with information about products stored in tables. Let's create an AFTER UPDATE trigger that tracks changes to the product price.
- Assume we have a table called Products with columns like ProductID, ProductName, and Price.
- The trigger will log any updates to the Price column into an auditing table (e.g., ProductAudit).
- Here's a simplified example of how you might create such a trigger:

```
CREATE TRIGGER ProductPriceUpdate

ON dbo.Products

AFTER UPDATE

AS

BEGIN

-- Insert the updated records into the audit table

INSERT INTO ProductAudit (ProductID, OldPrice, NewPrice, UpdatedDate)

SELECT i.ProductID, d.Price AS OldPrice, i.Price AS NewPrice, GETDATE()

FROM inserted i
```

```
JOIN deleted d ON i.ProductID = d.ProductID

WHERE i.Price <> d.Price;

END;
```

In this trigger:

- inserted and deleted are special tables available within triggers.
- We compare the old and new prices to identify changes.
- The GETDATE() function captures the timestamp of the update.
- Adjust the table and column names according to your schema

Additional Notes:

- Triggers can also be BEFORE triggers (executed before the DML statement) or INSTEAD OF triggers (overrides the original DML action).
- Keep triggers simple and avoid recursive or nested looping triggers for performance reasons.
- Remember to test thoroughly before deploying triggers in production environments.

b) Write a Trigger to check the age valid or not Using Message Alert.

Let's create a trigger that checks the age validity during an INSERT operation and raises an alert if the person's age is less than 18.

- In Oracle, you can use the RAISE_APPLICATION_ERROR procedure to raise an exception.
- Example

```
CREATE OR REPLACE TRIGGER trgCustomer

BEFORE INSERT ON customer

FOR EACH ROW

BEGIN

IF ADD_MONTHS(:new.cust_birthdate, 18 * 12) < SYSDATE THEN

RAISE_APPLICATION_ERROR(-20001, 'Customer must be at least 18 years old.');

END IF;
```

```
END;
```

- This trigger checks the customer's birthdate and raises an error if they are not at least 18 years old.

---

c) Create a Trigger to Raise appropriate error code and error

message.

Let's create a trigger that raises an appropriate error code and error message based on certain conditions.

- In Oracle, you can use the RAISE_APPLICATION_ERROR procedure to raise a custom error.
- **Example:**

```
CREATE OR REPLACE TRIGGER CheckExpiration
BEFORE INSERT ON CreditCard
FOR EACH ROW
BEGIN
IF ADD_MONTHS(TRUNC(SYSDATE), 12 * NEW.ExpYear + NEW.ExpMonth) < SYSDATE THEN
RAISE_APPLICATION_ERROR(-20001, 'The Credit Card you have entered has expired.');
END IF;
END;
```

- This trigger checks the expiration date and raises an error if the card has expired.

---

d) Create a Trigger on a table so that it will update another table

while insertingvalues.

Let's create a trigger that updates another table when inserting values into a primary table. I'll provide examples for both Oracle and SQL Server:

SQL Example:

- Suppose you have two tables: TableA and TableB.
- Whenever a new row is inserted into TableA, we want to populate TableB with relevant data.
- Here's an example of how you can achieve this using an AFTER INSERT trigger in Oracle:

```
CREATE OR REPLACE TRIGGER TableA_Trigger

AFTER INSERT ON TableA

FOR EACH ROW

BEGIN
```

```
INSERT INTO TableB (id, status, Flag, ResultOfAction, ExternalId, tableAId)

VALUES (:NEW.id, :NEW.status, :NEW.Flag, :NEW.ResultOfAction, :NEW.ExternalId, :NEW.id);

END;

/
```

In this trigger:

- :NEW represents the newly inserted row in TableA.
- We insert the relevant columns from TableA into TableB.

Week- 14

Write PL/SQL block for an application using cursors and all types of triggers.

Let's explore PL/SQL cursors and triggers. I'll provide examples for both cursors and different types of triggers.

1. **PL/SQL Cursors:**

- A cursor is a pointer that allows you to process rows returned by a query. PL/SQL has two types of cursors: implicit and explicit.
- **Implicit Cursors:**
  - Implicit cursors are automatically created by Oracle when executing SQL statements (e.g., SELECT INTO, INSERT, UPDATE, DELETE).
  - They provide information such as SQL%ROWCOUNT, SQL%ISOPEN, SQL%FOUND, and SQL%NOTFOUND.
  - However, they are not elegant when handling zero or multiple rows.
- **Explicit Cursors:**
  - Explicit cursors are explicitly declared in the PL/SQL code.
  - You have control over their execution cycle (OPEN, FETCH, CLOSE).
  - Here's an example of an explicit cursor:

2. In this example:

- We declare an explicit cursor named emp_cursor.
- We fetch rows into the emp_rec record type.
- The loop processes each row and displays employee information.

```
DECLARE

CURSOR emp_cursor IS

SELECT employee_id, first_name, last_name

FROM employees

WHERE department_id = 30;

emp_rec emp_cursor%ROWTYPE;

BEGIN

OPEN emp_cursor;

LOOP

FETCH emp_cursor INTO emp_rec;
```

```
EXIT WHEN emp_cursor%NOTFOUND;

DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_rec.employee_id || ', Name: ' || emp_rec.first_name || ' ' || emp_rec.last_name);

END LOOP;

CLOSE emp_cursor;

END;
```

PL/SQL Triggers:

- Triggers are predefined programs invoked automatically when specific events occur (e.g., BEFORE INSERT, AFTER UPDATE).
- They are stored in the database and executed repeatedly based on certain conditions.
- Here's an example of a BEFORE INSERT trigger:

```
CREATE OR REPLACE TRIGGER before_insert_example

BEFORE INSERT ON employees

FOR EACH ROW

BEGIN

IF :NEW.salary < 5000 THEN

RAISE_APPLICATION_ERROR(-20001, 'Salary must be at least 5000.');

END IF;

END;
```

- In this trigger:
  - We check if the salary being inserted is at least 5000.
  - If not, we raise a custom error.
- **Other Types of Triggers:**
  - Besides BEFORE INSERT, there are other types of triggers:
    - AFTER INSERT: Executes after an insert operation.
    - BEFORE UPDATE: Executes before an update operation.
    - AFTER DELETE: Executes after a delete operation.
    - You can create similar triggers for different scenarios.

Week- 14

Write a PL/SQL block for transaction operations of a typical application using package.

In a typical PL/SQL application, you can use transactions to group a set of related SQL statements together. Let's create a simple PL/SQL block that demonstrates transaction operations using COMMIT, ROLLBACK, and a package.

Explanation:

- We've created a package called order_mgmt_pkg with three procedures:

1. create_order: Inserts an order record into the "orders" table.
2. commit_transaction: Commits the transaction (makes changes permanent).
3. rollback_transaction: Rolls back the transaction (undoes changes).

Remember to adapt this example to your specific use case and modify the table and column names accordingly. ☺

-- Assume we have a table called "orders" with columns "order_id", "customer_id", and "total_amount".

-- Create a package specification (header) to define our transaction-related procedures.

```
CREATE OR REPLACE PACKAGE order_mgmt_pkg AS

PROCEDURE create_order(p_order_id NUMBER, p_customer_id NUMBER, p_total_amount NUMBER);

PROCEDURE commit_transaction;

PROCEDURE rollback_transaction;

END order_mgmt_pkg;
```

-- Create the package body to implement the procedures.

```
CREATE OR REPLACE PACKAGE BODY order_mgmt_pkg AS

PROCEDURE create_order(p_order_id NUMBER, p_customer_id NUMBER, p_total_amount NUMBER) IS

BEGIN
```

-- Insert an order record into the "orders" table.

```
INSERT INTO orders(order_id, customer_id, total_amount)
```

```
VALUES (p_order_id, p_customer_id, p_total_amount);

DBMS_OUTPUT.PUT_LINE('Order created successfully.');

END create_order;


PROCEDURE commit_transaction IS

BEGIN

COMMIT; -- Commit the transaction.

DBMS_OUTPUT.PUT_LINE('Transaction committed.');

END commit_transaction;


PROCEDURE rollback_transaction IS

BEGIN

ROLLBACK; -- Rollback the transaction.

DBMS_OUTPUT.PUT_LINE('Transaction rolled back.');

END rollback_transaction;

END order_mgmt_pkg;
```